

SIMATS ENGINEERING

ASSESSMENT TOOL 1- INDUSTRY PROBLEM SOLVING TASK

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO4	Assessment:	ASSESSMENT TOOL1- INDUSTRY PROBLEM SOLVING TASK
Weightage:	30%	Total Marks:	25
CO4 Statement:	<i>Implement hierarchical memory systems by differentiating cache levels (L1, L2, L3), virtual memory with paging, and advanced memory technologies (DRAM, SRAM, NAND Flash, MRAM, RRAM) based on latency, bandwidth, and throughput characteristics.</i>		
Student Name:	Patteela Yashwanth Kumar	Reg.No.:	192471055
Department:	CSE BIOSCIENCE	Date:	22-08-26

ANNEXURE A

INDUSTRY PROBLEM SOLVING TASK

1. Smartphone Performance Optimization

A mobile manufacturer observes lag while switching between apps. Analyze the role of L1, L2, L3 cache and DRAM and propose a hierarchical memory redesign to reduce latency and improve throughput.

2. Cloud Server Memory Bottleneck

A cloud service provider experiences high latency in database queries. Compare cache hierarchy vs virtual memory paging and recommend improvements using appropriate memory technologies.

3. Autonomous Vehicle Data Processing

An autonomous vehicle must process sensor data in real time. Evaluate SRAM, DRAM, and MRAM and design a memory hierarchy that maximizes bandwidth and minimizes latency.

4. AI Inference Edge Device

An edge AI device needs fast model loading with low power consumption. Compare NAND Flash, DRAM, and RRAM and recommend an optimized hierarchical memory structure.

5. High-Performance Gaming Console

A gaming console suffers frame drops during large scene loading. Analyze L3 cache vs DRAM throughput and propose memory hierarchy enhancements.

Code of Conduct:

I Patteela Yashwanth Kumar/192471055(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Patteela Yashwanth Kumar

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis

1. Smartphone Performance Optimization

Problem Analysis

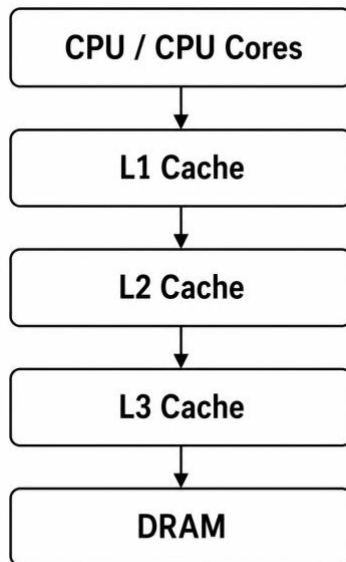
A smartphone manufacturer observes lag while switching between applications. The main reason can be frequent movement of data between the processor, cache memories, and DRAM. When the required application data is not available in cache, the processor must access slower DRAM, increasing memory access latency. The smartphone's memory system must efficiently handle **multiple applications running simultaneously** without causing delays. If frequently used application data is not available in the faster cache levels, the processor has to access DRAM more often, increasing access time. Therefore, an optimized memory hierarchy is required to **reduce latency, improve multitasking performance, and provide smooth application switching**.

Role of L1, L2, L3 Cache and DRAM

Memory Level	Role	Speed	Capacity	Main Purpose
L1 Cache	Closest to CPU. Stores the most frequently used instructions and data.	Very High (Lowest latency)	Smallest	Provides the fastest access to data that is immediately needed by the CPU.
L2 Cache	Larger than L1. Stores more frequently used data.	High (Higher than L1)	Larger than L1	Serves as a backup for L1 cache and reduces accesses to L3 and DRAM.
L3 Cache	Largest cache. Shared among CPU cores.	Moderate (Higher than L1/L2, lower than DRAM)	Largest Cache	Reduces the number of accesses to DRAM and improves multitasking performance.
DRAM	Main memory. Stores active applications and large datasets.	Lower (Highest latency compared to cache)	Very Large	Provides large storage capacity for active applications and system data.

- **L1 cache:** It is the smallest and fastest cache. It should contain frequently accessed instructions and data so that the processor can obtain them with minimum latency.
- **L2 cache:** It provides more capacity than L1. When data is absent from L1, the processor searches L2 before accessing slower memory.
- **L3 cache:** It is larger than L1 and L2 and can be shared among processor cores. It reduces the number of accesses to DRAM, especially during multitasking.
- **DRAM:** DRAM provides large memory capacity for running applications. However, it has considerably higher access latency than cache.

Proposed Hierarchical Memory Design



Recommended Improvements

1. Increase L2/L3 cache capacity to reduce DRAM accesses.
2. Use efficient cache replacement policies.
3. Keep frequently used application data in cache.
4. Use high-bandwidth DRAM for multitasking.
5. Use prefetching to load frequently required data before it is requested.
6. Optimize the operating system's memory management to reduce unnecessary data movement.

Analytical Result

The redesigned hierarchy provides **low latency through L1/L2**, better shared-data handling through **L3**, and sufficient capacity through **DRAM**. Therefore, the number of expensive DRAM accesses decreases, improving application switching speed and overall smartphone throughput.

Conclusion

A balanced L1–L2–L3–DRAM hierarchy is the most practical solution. Cache handles frequently accessed data while DRAM provides large-capacity storage, thereby reducing application-switching latency.

2. Cloud Server Memory Bottleneck

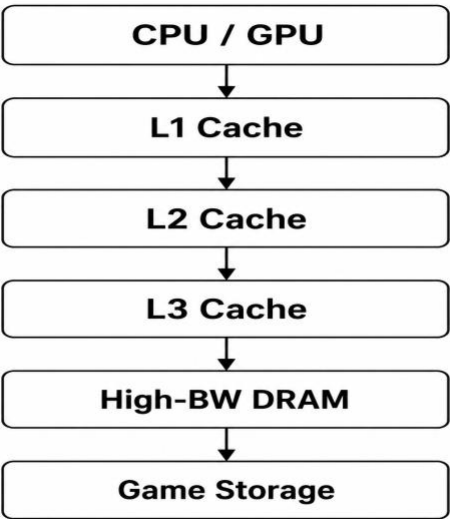
Problem Analysis

A cloud service provider experiences high latency during database queries. Database applications frequently access the same instructions and data. If useful data is available in cache, it can be accessed quickly. If it is not available, the system may access DRAM or, in a virtual-memory situation, secondary storage through paging. Therefore, excessive cache misses and paging can significantly increase query latency.

Cache Hierarchy vs Virtual Memory Paging

Feature	Cache Hierarchy	Virtual Memory Paging
Purpose	Reduce memory access latency	Extend available memory
Levels	L1, L2, L3	Pages in virtual/physical memory
Speed	Very fast	Slower during page faults
Managed by	Hardware	OS + MMU
Main issue	Cache miss	Page fault
Best use	Frequently accessed data	Memory capacity management

Cache Hierarchy



Virtual Memory Paging

- Virtual memory allows processes to use an address space larger than physical DRAM. Memory is divided into pages. If the required page is not in physical memory, a **page fault** occurs and the operating system has to retrieve it from secondary storage.
- This process is much slower than a cache access and can cause severe database-query latency when excessive paging occurs.

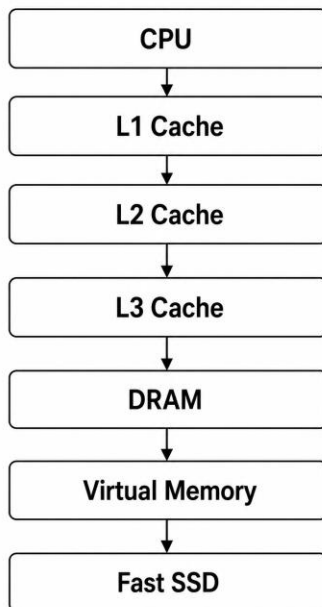
Recommended Memory Technologies and Improvements

1. Increase DRAM capacity so that active database working sets remain in physical memory.
2. Improve L3 cache capacity to reduce processor-to-DRAM traffic.
3. Use high-bandwidth DRAM for database servers.
4. Minimize page faults by allocating sufficient physical memory.
5. Use efficient database buffer/cache management.
6. Use fast storage such as SSDs when paging or persistent data access is unavoidable.
7. Monitor cache-hit ratio, DRAM utilization, page-fault rate, and query latency using server performance-monitoring tools.

Analytical Evaluation

For database queries, **cache optimization should be the first priority** because cache access is much faster than DRAM. Virtual memory is useful for capacity management but should not be relied upon for performance-critical database working sets.

A suitable architecture is:



Conclusion

The cloud provider should prioritize cache efficiency and adequate DRAM capacity. Virtual-memory paging should be minimized because frequent page faults can produce very high latency. A larger cache hierarchy combined with sufficient high-bandwidth DRAM provides better database performance.

3. Autonomous Vehicle Data Processing

Problem Analysis

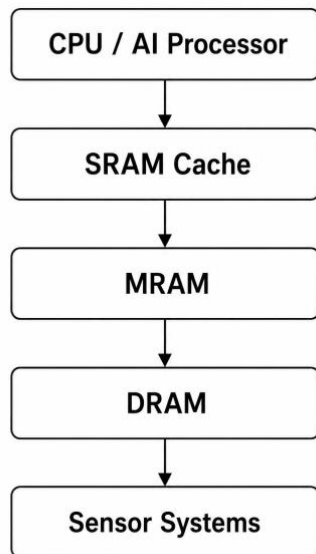
An autonomous vehicle continuously receives data from cameras, LiDAR, radar, ultrasonic sensors, GPS, and other systems. This data must be processed in real time. Therefore, the memory system must provide **very low latency and high bandwidth**. Autonomous vehicles generate a continuous stream of data from **cameras, LiDAR, radar, GPS, and other sensors**. This data must be processed quickly to support real-time decisions such as object detection, obstacle avoidance, and navigation. Any delay in memory access can affect vehicle response time and safety. Therefore, an efficient memory hierarchy using **SRAM, DRAM, and MRAM** is required to provide **low latency, high bandwidth, sufficient capacity, and reliable data handling**.

Evaluation of SRAM, DRAM and MRAM

Feature	SRAM	DRAM	MRAM
Speed	Very High	High	High
Latency	Very Low	Higher than SRAM	Low
Capacity	Low	High	Medium
Volatility	Volatile	Volatile	Non-volatile
Refresh	Not required	Required	Not required
Main Advantage	Fastest access	Large capacity	Fast + non-volatile
Suitable Use	CPU/AI cache	Main memory	Critical & persistent data storage

- **SRAM:** SRAM provides very fast access and does not require periodic refresh. Its major limitation is lower density and higher cost per bit. It is therefore suitable for processor caches.
- **DRAM:** DRAM provides high capacity at relatively low cost and is suitable for storing large sensor datasets and active processing data.
- **MRAM:** MRAM is non-volatile and provides fast access with no refresh requirement. It can be useful for retaining important data and supporting fast local storage.

Proposed Memory Hierarchy



Recommended Design

- **SRAM layer:** Store the most frequently accessed sensor features, instructions, and control data.
- **DRAM layer:** Store large real-time sensor frames, intermediate AI-processing data, maps, and active application datasets.
- **MRAM layer:** Store critical parameters, frequently reused data, configuration information, and selected persistent data where fast non-volatility is useful.

Engineering Improvements

1. Use large, fast SRAM caches near processing cores.
2. Use high-bandwidth DRAM for camera and LiDAR workloads.
3. Use MRAM for fast non-volatile data retention.
4. Use parallel memory channels to increase bandwidth.
5. Use data prefetching for predictable sensor streams.
6. Partition memory according to real-time priority.
7. Keep safety-critical processing data in the lowest-latency memory available.

Analytical Result

Real-time autonomous driving requires predictable response rather than simply maximum memory capacity. SRAM minimizes latency for critical processing, DRAM provides the capacity needed for large sensor workloads, and MRAM provides fast non-volatile storage.

Conclusion

A **SRAM** → **MRAM** → **DRAM** hierarchy provides a good balance between latency, bandwidth, capacity, and persistence. SRAM should handle the most time-critical data, DRAM should handle large active datasets, and MRAM should support fast non-volatile requirements.

4. AI Inference Edge Device

Problem Analysis

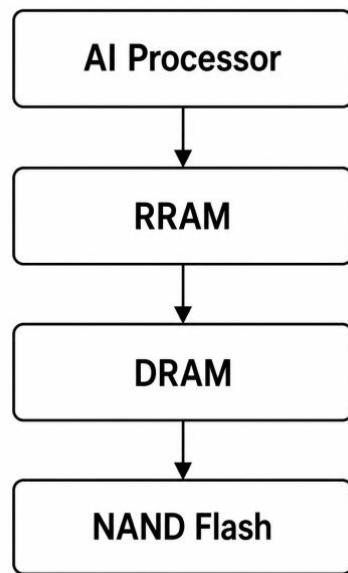
An edge AI device needs to load machine-learning models quickly while consuming low power. The memory system must therefore balance **storage capacity, loading speed, latency, and energy consumption**. **AI inference quickly while operating with limited power**. Large AI models require sufficient storage, while active inference needs fast memory access. Frequent movement of model data between storage and working memory can increase **latency and power consumption**. Therefore, an optimized memory hierarchy using **NAND Flash, DRAM, and RRAM** is required to achieve **fast model loading, efficient processing, low power consumption, and reliable storage**. The three technologies considered are NAND Flash, DRAM, and RRAM.

Comparison

Feature	NAND Flash	DRAM	RRAM
Nature	Non-volatile	Volatile	Non-volatile
Speed	Lower	High	Fast
Capacity	Very High	High	Medium
Latency	Higher	Low	Low
Power	Low for storage	Higher	Low
Main Advantage	High-capacity storage	Fast working memory	Fast + non-volatile
Suitable Use	AI model storage	Active model execution	Frequently accessed model data

- **NAND Flash:** Provides high-density non-volatile storage. It is suitable for permanently storing AI models but is slower than DRAM for active processing.
- **DRAM:** Provides fast temporary memory required while an AI model is executing. However, it is volatile and consumes power for operation and refresh.
- **RRAM:** Resistive RAM is a non-volatile technology that can provide fast access and potentially lower energy consumption. It is useful for edge devices where power efficiency is important.

Optimized Hierarchical Structure



Working Principle

1. AI models are permanently stored in **NAND Flash**.
2. When an inference task starts, the required model is transferred to **DRAM**.
3. Frequently accessed model parameters or selected data can be placed in **RRAM**.
4. The processor accesses the faster memory during inference.
5. Unused models remain in NAND Flash to reduce active memory usage.

Recommended Improvements

- Use NAND Flash for high-capacity model storage.
- Use DRAM for active inference workloads.
- Use RRAM for frequently accessed parameters and fast non-volatile storage.
- Reduce unnecessary data movement between memory levels.
- Apply model compression or quantization where appropriate to reduce memory requirements.
- Use power-aware memory management to deactivate unused memory resources.

Analytical Result

NAND Flash provides capacity but has higher access latency. DRAM provides fast working memory but is volatile. RRAM provides an attractive intermediate layer because of its non-volatility and fast-access characteristics.

Conclusion

The recommended hierarchy is **NAND Flash** → **DRAM** → **RRAM**, with frequently accessed information moved toward the faster memory levels. This structure supports rapid model loading while reducing unnecessary power consumption.

5. High-Performance Gaming Console

Problem Analysis

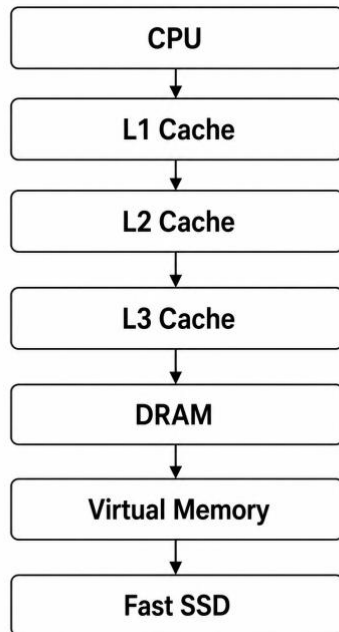
A gaming console experiences frame drops while loading large scenes. Modern games continuously transfer large textures, models, maps, audio files, and other assets. If memory cannot supply these assets at sufficient speed, the processor/GPU may wait for data, causing frame drops. A high-performance gaming console must process **large game scenes, high-resolution textures, character models, and other assets** without causing frame drops or delays. If the required data is not available quickly, the CPU/GPU may remain idle while waiting for memory access. Therefore, an optimized memory hierarchy using **L3 cache and high-bandwidth DRAM** is required to reduce latency, increase data-transfer speed, and provide smooth gaming performance.

L3 Cache vs DRAM

Feature	L3 Cache	DRAM
Capacity	Smaller	Much larger
Latency	Very Low	Higher than cache
Speed	Very High	High
Cost per bit	High	Lower
Main Purpose	Frequently reused data	Large game assets
Main Advantage	Fast access	High capacity & bandwidth

- **L3 Cache:** L3 provides fast access to frequently used instructions and data. It reduces the number of accesses to DRAM but cannot store an entire large game scene.
- **DRAM:** DRAM provides the large capacity required for textures, game assets, and other large datasets. High-bandwidth DRAM is particularly important for GPU-intensive workloads.

Proposed Memory Hierarchy



L3 Cache Enhancement

The console should use a sufficiently large L3 cache to retain:

- Frequently used game instructions
- Repeatedly accessed game data
- Frequently used rendering information
- CPU/GPU coordination data

This reduces repeated DRAM accesses.

DRAM Enhancement

The DRAM subsystem should provide:

1. High memory bandwidth.
2. Multiple memory channels where supported.
3. Efficient data transfer between CPU/GPU and memory.
4. Sufficient capacity for large game scenes.
5. Efficient memory scheduling.

Analytical Result

L3 cache has much lower latency but limited capacity. DRAM has higher latency but provides much greater capacity and can deliver high bandwidth when properly designed. Therefore, increasing L3 alone cannot solve large-scene loading problems.

The best solution is to combine an appropriately sized L3 cache with **high-bandwidth DRAM** and efficient prefetching/data-transfer mechanisms.

Conclusion

Frame drops can be reduced by improving both cache efficiency and DRAM bandwidth. L3 should handle frequently reused data, while DRAM should handle large scene assets. A balanced hierarchy prevents the CPU/GPU from waiting excessively for memory and improves gaming performance.

